

LABORATORY MANUAL

For

DATABASE MANAGEMENT SYSTEMS

Course Code: U25DSPC307

S.Y. BTECH

SEMESTER-III

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL NO. 4

MULTI-TABLE JOIN QUERIES

1. Aim

To retrieve data from multiple related tables using INNER JOIN, OUTER JOIN (LEFT, RIGHT and FULL), SELF JOIN and CROSS JOIN in MySQL.

2. Problem Statement

Using the *college_db* sample database, write join queries to:

1. List every student with the code and name of his/her department.
2. List the students enrolled in Database Management Systems (CS301) with marks and grade.
3. Show the number of students in every department, including departments with no students, and list students who have not enrolled in any course.
4. Show every course with its instructor, including unassigned courses, and list courses in which no student has enrolled.
5. Show all instructors and all courses together (full outer join).
6. Show each instructor with the name of his/her mentor, and pairs of students who live in the same city.
7. Generate all combinations of lab batches (B1-B3) and lab slots for a practical timetable.
8. Use USING, NATURAL JOIN and a non-equi join with a grade scale table.

Note: The course document lists only the title of this practical. The case study, sample data and individual tasks below were prepared for this report to fulfil that title; they are illustrative and are not official university questions. All names, e-mail addresses and phone numbers are fictitious.

3. Objectives

1. To understand why joins are required in a normalized database.
2. To write inner joins over two and three tables.
3. To use left and right outer joins to include unmatched rows and to find missing data.
4. To emulate FULL OUTER JOIN in MySQL using UNION.
5. To use self joins for recursive relationships and cross joins for combinations.

4. Software / Tools Required

Sr. No.	Software / Tool	Purpose
1	MySQL Community Server 8.0 (version 8.0.46 used)	Relational DBMS on which all SQL statements were executed
2	MySQL command-line client (mysql)	Running the SQL scripts and recording the output
3	Operating system: Ubuntu 24.04 LTS (Windows 10/11 also suitable)	Platform for the DBMS server and client
4	Text editor (VS Code / Notepad++)	Writing and saving the .sql script files

5. Theory

5.1 Need for Joins

Normalization stores related facts in different tables (students, courses and enrollments). A **join** combines rows of two or more tables, usually by matching a foreign key with the primary key it references, so that the related facts can be seen together in one result. Conceptually a join is a Cartesian product followed by a selection of the matching rows.

5.2 Types of Joins

Join type	Rows returned	MySQL syntax
INNER JOIN (equi-join)	Only rows that have a match in both tables	A [INNER] JOIN B ON A.x = B.x
LEFT OUTER JOIN	All rows of the left table; NULLs for the right table when no match	A LEFT [OUTER] JOIN B ON ...
RIGHT OUTER JOIN	All rows of the right table; NULLs for the left table when no match	A RIGHT [OUTER] JOIN B ON ...
FULL OUTER JOIN	All rows of both tables, matched where possible	Not supported - LEFT JOIN UNION RIGHT JOIN
SELF JOIN	A table joined with itself using two aliases	T a JOIN T b ON a.x = b.y
CROSS JOIN	Cartesian product: every row of A with every row of B ($ A \times B $ rows)	A CROSS JOIN B
NATURAL JOIN	Equi-join on all columns having the same name	A NATURAL JOIN B
Non-equi (theta) join	Join condition uses <, >, BETWEEN etc. instead of =	A JOIN B ON A.m BETWEEN B.lo AND B.hi

5.3 Important Points

- Table aliases (student s) shorten queries and are compulsory in a self join.
- In an outer join, a condition on the inner table placed in WHERE removes the NULL rows; WHERE right.key IS NULL is used deliberately to find unmatched rows (anti-join).
- JOIN ... USING (col) joins on a column with the same name in both tables and shows it once. NATURAL JOIN is risky because any newly added column with a common name silently changes the join condition.
- MySQL treats JOIN, CROSS JOIN and INNER JOIN as equivalent; a CROSS JOIN with an ON clause behaves as an inner join.
- PostgreSQL, SQL Server and Oracle support FULL OUTER JOIN directly; in MySQL it must be emulated with UNION of a LEFT and a RIGHT join.

5.4 Relational Algebra View

INNER JOIN corresponds to the theta-join $R \bowtie_{\text{condition}} S = \sigma_{\text{condition}}(R \times S)$, CROSS JOIN to the Cartesian product $R \times S$, and NATURAL JOIN to the natural join operator of relational algebra.

6. Procedure / Algorithm

1. Create *college_db* and load the sample data (Q1, Q2); create the small tables lab_batch, lab_slot and grade_scale (Q3).
2. Identify the primary key - foreign key pairs that link the tables: student.dept_id - department.dept_id, enrollment.student_id - student.student_id, enrollment.course_id - course.course_id, course.instr_id - instructor.instr_id and instructor.mentor_id - instructor.instr_id.
3. Write INNER JOIN queries for two and three tables.
4. Write LEFT and RIGHT OUTER JOIN queries, including anti-joins that find unmatched rows.
5. Attempt FULL OUTER JOIN, observe the error and emulate it using UNION.
6. Write SELF JOIN queries using two aliases of the same table.
7. Write a CROSS JOIN and verify that the number of rows equals the product of the row counts.
8. Write queries using USING, NATURAL JOIN and a non-equi join.
9. Record and verify the output of each query against the sample data.

7. SQL Queries / Program

Q1: Setup: create college_db and its tables (from Practical 2)

```
DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE department (
    dept_id      INT          NOT NULL,
    dept_code    VARCHAR(10)  NOT NULL,
    dept_name    VARCHAR(40)  NOT NULL,
    building     VARCHAR(20),
    budget       DECIMAL(12,2) DEFAULT 1000000.00,
    CONSTRAINT pk_department PRIMARY KEY (dept_id),
    CONSTRAINT uq_dept_code  UNIQUE (dept_code),
    CONSTRAINT chk_budget    CHECK (budget > 0)
);

CREATE TABLE instructor (
    instr_id      INT          NOT NULL,
    instr_name    VARCHAR(40)  NOT NULL,
    designation    VARCHAR(20)  NOT NULL,
    dept_id       INT,
    salary        DECIMAL(10,2) NOT NULL,
    email         VARCHAR(40)  NOT NULL,
    mentor_id     INT,
    CONSTRAINT pk_instructor  PRIMARY KEY (instr_id),
    CONSTRAINT uq_instr_email UNIQUE (email),
```

```

CONSTRAINT chk_salary      CHECK (salary BETWEEN 30000 AND 300000),
CONSTRAINT fk_instr_dept   FOREIGN KEY (dept_id)
    REFERENCES department (dept_id) ON DELETE SET NULL ON UPDATE CASCADE,
CONSTRAINT fk_instr_mentor FOREIGN KEY (mentor_id)
    REFERENCES instructor (instr_id) ON DELETE SET NULL
);

CREATE TABLE student (
    student_id INT NOT NULL AUTO_INCREMENT,
    roll_no    VARCHAR(10) NOT NULL,
    student_name VARCHAR(40) NOT NULL,
    gender     CHAR(1) NOT NULL,
    dob        DATE NOT NULL,
    dept_id    INT NOT NULL,
    city       VARCHAR(30) DEFAULT 'Pune',
    email      VARCHAR(40) NOT NULL,
    CONSTRAINT pk_student PRIMARY KEY (student_id),
    CONSTRAINT uq_roll_no  UNIQUE (roll_no),
    CONSTRAINT uq_student_email UNIQUE (email),
    CONSTRAINT chk_gender  CHECK (gender IN ('M', 'F', 'O')),
    CONSTRAINT chk_dob     CHECK (dob >= '1995-01-01'),
    CONSTRAINT fk_student_dept FOREIGN KEY (dept_id)
    REFERENCES department (dept_id) ON UPDATE CASCADE
);

CREATE TABLE student_phone (
    student_id INT NOT NULL,
    phone      CHAR(10) NOT NULL,
    CONSTRAINT pk_student_phone PRIMARY KEY (student_id, phone),
    CONSTRAINT chk_phone        CHECK (phone REGEXP '^[6-9][0-9]{9}$'),
    CONSTRAINT fk_phone_student FOREIGN KEY (student_id)
    REFERENCES student (student_id) ON DELETE CASCADE
);

CREATE TABLE course (
    course_id CHAR(5) NOT NULL,
    title     VARCHAR(40) NOT NULL,
    dept_id   INT NOT NULL,
    credits   TINYINT NOT NULL,
    instr_id  INT,
    prereq_id CHAR(5),
    CONSTRAINT pk_course PRIMARY KEY (course_id),
    CONSTRAINT uq_course_title UNIQUE (title),
    CONSTRAINT chk_credits CHECK (credits BETWEEN 1 AND 5),
    CONSTRAINT fk_course_dept FOREIGN KEY (dept_id)
    REFERENCES department (dept_id) ON UPDATE CASCADE,
    CONSTRAINT fk_course_instr FOREIGN KEY (instr_id)
    REFERENCES instructor (instr_id) ON DELETE SET NULL,
    CONSTRAINT fk_course_prereq FOREIGN KEY (prereq_id)
    REFERENCES course (course_id)
);

CREATE TABLE enrollment (
    student_id INT NOT NULL,
    course_id  CHAR(5) NOT NULL,
    semester   TINYINT NOT NULL,
    marks      INT,
    grade      CHAR(2),
    CONSTRAINT pk_enrollment PRIMARY KEY (student_id, course_id),
    CONSTRAINT chk_semester CHECK (semester BETWEEN 1 AND 8),
    CONSTRAINT chk_marks    CHECK (marks BETWEEN 0 AND 100),
    CONSTRAINT fk_enr_student FOREIGN KEY (student_id)
    REFERENCES student (student_id) ON DELETE CASCADE,
    CONSTRAINT fk_enr_course FOREIGN KEY (course_id) REFERENCES course (course_id)
);

```

This setup script recreates the common *college_db* case-study database (schema of Practical 2 and data of Practical 3) so that this practical can be executed independently on a fresh MySQL

server.

Q2: Setup: insert the sample data (from Practical 3)

```

INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES
  (10, 'CSE', 'Computer Engineering', 'A Block', 2500000),
  (20, 'AIML', 'AI and Machine Learning', 'B Block', 3000000),
  (30, 'ECE', 'Electronics and Telecomm', 'C Block', 1800000),
  (40, 'MECH', 'Mechanical Engineering', 'D Block', 1500000),
  (50, 'CIVIL', 'Civil Engineering', 'D Block', 1200000);

INSERT INTO instructor VALUES
  (101, 'Anil Kulkarni', 'Professor', 10, 185000, 'anil.k@example.com', NULL),
  (102, 'Meera Joshi', 'Professor', 20, 172000, 'meera.j@example.com', 101),
  (103, 'Rahul Patil', 'Asst. Prof', 10, 82000, 'rahul.p@example.com', 101),
  (104, 'Sneha Deshmukh', 'Assoc. Prof', 20, 115000, 'sneha.d@example.com', 102),
  (105, 'Vikram Shinde', 'Professor', 30, 168000, 'vikram.s@example.com', 101),
  (106, 'Pooja Kale', 'Asst. Prof', 20, 78000, 'pooja.k@example.com', 104),
  (107, 'Nitin Pawar', 'Asst. Prof', 50, 76000, 'nitin.p@example.com', 105),
  (108, 'Kavita More', 'Assoc. Prof', 40, 112000, 'kavita.m@example.com', 105);

INSERT INTO student
  (roll_no, student_name, gender, dob, dept_id, city, email) VALUES
  ('S24CS001', 'Aarav Sharma', 'M', '2005-03-14', 10, 'Pune', 'aarav@example.com'),
  ('S24CS002', 'Isha Gupta', 'F', '2005-07-22', 10, 'Mumbai', 'isha@example.com'),
  ('S24AI001', 'Rohan Mehta', 'M', '2005-01-09', 20, 'Pune', 'rohan@example.com'),
  ('S24AI002', 'Priya Nair', 'F', '2004-11-30', 20, 'Nashik', 'priya@example.com'),
  ('S24AI003', 'Kunal Verma', 'M', '2005-05-18', 20, 'Pune', 'kunal@example.com'),
  ('S24EC001', 'Sneha Iyer', 'F', '2005-09-02', 30, 'Nagpur', 'sneha@example.com'),
  ('S24EC002', 'Aditya Rao', 'M', '2004-12-25', 30, 'Mumbai', 'aditya@example.com'),
  ('S24ME001', 'Neha Kulkarni', 'F', '2005-02-11', 40, 'Pune', 'neha@example.com'),
  ('S24CS003', 'Yash Jadhav', 'M', '2005-08-05', 10, NULL, 'yash@example.com'),
  ('S24AI004', 'Ananya Desai', 'F', '2005-04-27', 20, 'Mumbai', 'ananya@example.com');

INSERT INTO student_phone VALUES
  (1, '9876500011'), (1, '9876500012'), (2, '9876500021'), (3, '9876500031'),
  (4, '9876500041'), (4, '9876500042'), (6, '9876500061'), (9, '9876500091');

INSERT INTO course VALUES
  ('CS101', 'Programming in C', 10, 4, 103, NULL),
  ('CS201', 'Data Structures', 10, 4, 103, 'CS101'),
  ('CS301', 'Database Management Systems', 10, 3, 101, 'CS201'),
  ('CS401', 'Advanced Databases', 10, 3, 101, 'CS301'),
  ('AI201', 'Python for Data Science', 20, 3, 104, 'CS101'),
  ('AI301', 'Machine Learning', 20, 4, 102, 'AI201'),
  ('AI401', 'Deep Learning', 20, 3, 106, 'AI301'),
  ('EC201', 'Digital Electronics', 30, 3, 105, NULL),
  ('EC301', 'Signals and Systems', 30, 4, NULL, 'EC201'),
  ('ME201', 'Engineering Mechanics', 40, 3, 108, NULL);

INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES
  (1, 'CS101', 3, 78, 'A'), (1, 'CS201', 3, 85, 'A+'), (1, 'CS301', 4, 91, 'O'),
  (1, 'AI201', 3, 72, 'A'), (2, 'CS101', 3, 88, 'A+'), (2, 'CS201', 3, 67, 'B+'),
  (2, 'CS301', 4, 74, 'A'), (3, 'CS101', 3, 65, 'B+'), (3, 'AI201', 3, 81, 'A+'),
  (3, 'AI301', 4, 89, 'A+'), (3, 'CS301', 4, 70, 'A'), (4, 'AI201', 3, 92, 'O'),
  (4, 'AI301', 4, 76, 'A'), (4, 'AI401', 4, 84, 'A+'), (5, 'AI201', 3, 58, 'B'),
  (5, 'AI301', 4, 63, 'B+'), (6, 'EC201', 3, 79, 'A'), (6, 'CS101', 3, 55, 'B'),
  (6, 'EC301', 4, 82, 'A+'), (7, 'EC201', 3, 45, 'C'), (7, 'EC301', 4, 61, 'B+'),
  (8, 'CS101', 3, 78, 'A'), (9, 'CS201', 3, 90, 'O'), (9, 'CS301', 4, 95, 'O'),
  (9, 'CS401', 4, 88, 'A+');

```

Q3: Setup: small tables for CROSS JOIN and non-equi join

```
CREATE TABLE lab_batch (batch CHAR(2) PRIMARY KEY);
CREATE TABLE lab_slot (slot_day CHAR(3), slot_time VARCHAR(11),
                        PRIMARY KEY (slot_day, slot_time));
CREATE TABLE grade_scale (grade CHAR(2) PRIMARY KEY, min_marks INT, max_marks INT);
INSERT INTO lab_batch VALUES ('B1'), ('B2'), ('B3');
INSERT INTO lab_slot VALUES ('Mon', '10:00-12:00'), ('Wed', '14:00-16:00');
INSERT INTO grade_scale VALUES ('O', 90, 100), ('A+', 80, 89), ('A', 70, 79),
                                ('B+', 60, 69), ('B', 50, 59), ('C', 40, 49), ('F', 0, 39);
```

Q4: INNER JOIN of two tables: students with their department

```
SELECT s.roll_no, s.student_name, d.dept_code, d.dept_name
FROM   student s
INNER JOIN department d ON s.dept_id = d.dept_id
ORDER BY d.dept_code, s.roll_no;
```

Q5: INNER JOIN of three tables: marks of students in Database Management Systems

```
SELECT s.roll_no, s.student_name, c.title, e.marks, e.grade
FROM   enrollment e
JOIN   student s ON s.student_id = e.student_id
JOIN   course c ON c.course_id = e.course_id
WHERE  c.course_id = 'CS301'
ORDER BY e.marks DESC;
```

Q6: LEFT OUTER JOIN: all departments with their student count; students with no enrollment

```
SELECT d.dept_code, COUNT(s.student_id) AS no_of_students
FROM   department d
LEFT JOIN student s ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_code
ORDER BY d.dept_id;

SELECT s.roll_no, s.student_name, e.course_id
FROM   student s
LEFT JOIN enrollment e ON e.student_id = s.student_id
WHERE  e.student_id IS NULL;
```

Q7: RIGHT OUTER JOIN: every course with its instructor (unassigned courses included)

```
SELECT c.course_id, c.title, i.instr_name
FROM   instructor i
RIGHT JOIN course c ON c.instr_id = i.instr_id
ORDER BY c.course_id;

SELECT c.course_id, c.title
FROM   enrollment e
RIGHT JOIN course c ON c.course_id = e.course_id
WHERE  e.course_id IS NULL;
```

Q8: FULL OUTER JOIN: not supported by MySQL, emulated with UNION

```

SELECT i.instr_name, c.course_id
FROM   instructor i FULL OUTER JOIN course c ON c.instr_id = i.instr_id;

SELECT i.instr_name, c.course_id
FROM   instructor i LEFT JOIN course c ON c.instr_id = i.instr_id
UNION
SELECT i.instr_name, c.course_id
FROM   instructor i RIGHT JOIN course c ON c.instr_id = i.instr_id
ORDER BY instr_name IS NULL, instr_name, course_id;

```

MySQL does not implement FULL OUTER JOIN, so the first statement is rejected with a syntax error (ERROR 1064). The second statement combines the LEFT and RIGHT joins with UNION (which also removes the duplicate matched rows) and returns matched rows, instructors without courses and courses without instructors.

Q9: SELF JOIN: instructor with mentor; pairs of students from the same city

```

SELECT i.instr_name AS instructor, i.designation,
       m.instr_name AS mentor
FROM   instructor i
LEFT JOIN instructor m ON i.mentor_id = m.instr_id
ORDER BY i.instr_id;

SELECT a.student_name AS student_1, b.student_name AS student_2, a.city
FROM   student a
JOIN   student b ON a.city = b.city AND a.student_id < b.student_id
ORDER BY a.city, student_1;

```

Q10: CROSS JOIN: all lab batch and time-slot combinations

```

SELECT b.batch, s.slot_day, s.slot_time
FROM   lab_batch b
CROSS JOIN lab_slot s
ORDER BY b.batch, s.slot_day;

```

Q11: Other forms: JOIN ... USING, NATURAL JOIN and a non-equi (theta) join

```

SELECT student_id, course_id, title, marks
FROM   enrollment JOIN course USING (course_id)
WHERE  student_id = 4;

SELECT dept_id, dept_code, course_id, title
FROM   department NATURAL JOIN course
WHERE  dept_code = 'ECE';

SELECT s.student_name, e.course_id, e.marks, g.grade AS computed_grade
FROM   enrollment e
JOIN   student s   ON s.student_id = e.student_id
JOIN   grade_scale g ON e.marks BETWEEN g.min_marks AND g.max_marks
WHERE  e.course_id = 'CS101'
ORDER BY e.marks DESC;

```

8. Sample Input / Database Tables

The following tables (created by Q1-Q3) are used in this practical. The values were read from the database after executing the setup queries.

Table: department

dept_id	dept_code	dept_name	building	budget
10	CSE	Computer Engineering	A Block	2500000.00
20	AIML	AI and Machine Learning	B Block	3000000.00
30	ECE	Electronics and Telecomm	C Block	1800000.00
40	MECH	Mechanical Engineering	D Block	1500000.00
50	CIVIL	Civil Engineering	D Block	1200000.00

Table: instructor

instr_id	instr_name	designation	dept_id	salary	email	mentor_id
101	Anil Kulkarni	Professor	10	185000.00	anil.k@example.com	NULL
102	Meera Joshi	Professor	20	172000.00	meera.j@example.com	101
103	Rahul Patil	Asst. Prof	10	82000.00	rahul.p@example.com	101
104	Sneha Deshmukh	Assoc. Prof	20	115000.00	sneha.d@example.com	102
105	Vikram Shinde	Professor	30	168000.00	vikram.s@example.com	101
106	Pooja Kale	Asst. Prof	20	78000.00	pooja.k@example.com	104
107	Nitin Pawar	Asst. Prof	50	76000.00	nitin.p@example.com	105
108	Kavita More	Assoc. Prof	40	112000.00	kavita.m@example.com	105

Table: student

student_id	roll_no	student_name	gender	dob	dept_id	city	email
1	S24CS001	Aarav Sharma	M	2005-03-14	10	Pune	aarav@example.com
2	S24CS002	Isha Gupta	F	2005-07-22	10	Mumbai	isha@example.com
3	S24AI001	Rohan Mehta	M	2005-01-09	20	Pune	rohan@example.com
4	S24AI002	Priya Nair	F	2004-11-30	20	Nashik	priya@example.com
5	S24AI003	Kunal Verma	M	2005-05-18	20	Pune	kunal@example.com
6	S24EC001	Sneha Iyer	F	2005-09-02	30	Nagpur	sneha@example.com
7	S24EC002	Aditya Rao	M	2004-12-25	30	Mumbai	aditya@example.com
8	S24ME001	Neha Kulkarni	F	2005-02-11	40	Pune	neha@example.com
9	S24CS003	Yash Jadhav	M	2005-08-05	10	NULL	yash@example.com
10	S24AI004	Ananya Desai	F	2005-04-27	20	Mumbai	ananya@example.com

Table: course

course_id	title	dept_id	credits	instr_id	prereq_id
AI201	Python for Data Science	20	3	104	CS101
AI301	Machine Learning	20	4	102	AI201
AI401	Deep Learning	20	3	106	AI301
CS101	Programming in C	10	4	103	NULL
CS201	Data Structures	10	4	103	CS101
CS301	Database Management Systems	10	3	101	CS201
CS401	Advanced Databases	10	3	101	CS301
EC201	Digital Electronics	30	3	105	NULL
EC301	Signals and Systems	30	4	NULL	EC201
ME201	Engineering Mechanics	40	3	108	NULL

Table: enrollment (25 rows)

student_id	course_id	semester	marks	grade
1	AI201	3	72	A
1	CS101	3	78	A
1	CS201	3	85	A+
1	CS301	4	91	O
2	CS101	3	88	A+
2	CS201	3	67	B+
2	CS301	4	74	A
3	AI201	3	81	A+
3	AI301	4	89	A+
3	CS101	3	65	B+
3	CS301	4	70	A
4	AI201	3	92	O
4	AI301	4	76	A
4	AI401	4	84	A+
5	AI201	3	58	B
5	AI301	4	63	B+
6	CS101	3	55	B
6	EC201	3	79	A
6	EC301	4	82	A+
7	EC201	3	45	C
7	EC301	4	61	B+
8	CS101	3	78	A
9	CS201	3	90	O
9	CS301	4	95	O
9	CS401	4	88	A+

Tables: lab_batch and lab_slot (Cartesian product)

batch	slot_day	slot_time
B1	Mon	10:00-12:00
B1	Wed	14:00-16:00

Table: grade_scale

grade	min_marks	max_marks
O	90	100
A+	80	89
A	70	79
B+	60	69
B	50	59
C	40	49
F	0	39

9. Output

The output below is the actual text produced by the MySQL 8.0.46 command-line client when the statements of Section 7 were executed in batch mode (`mysql --table -vvv < script.sql`). Each result is preceded by the statement that produced it; statements longer than a few lines are abbreviated with '...'. Because the script ran in batch mode, error messages report the line number within the script file. Execution times shown in brackets depend on the machine.

Output of Q1: Setup: create college_db and its tables (from Practical 2)

```
mysql> DROP DATABASE IF EXISTS college_db;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.00 sec)

mysql> CREATE TABLE department ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE instructor ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE student_phone ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE course ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE enrollment ( ...;
Query OK, 0 rows affected (0.02 sec)
```

Output of Q2: Setup: insert the sample data (from Practical 3)

```
mysql> INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES ...;
Query OK, 5 rows affected (0.00 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> INSERT INTO instructor VALUES ...;
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student_phone VALUES ...;
Query OK, 8 rows affected (0.00 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO course VALUES ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES ...;
Query OK, 25 rows affected (0.00 sec)
Records: 25 Duplicates: 0 Warnings: 0
```

Output of Q3: Setup: small tables for CROSS JOIN and non-equi join

```
mysql> CREATE TABLE lab_batch (batch CHAR(2) PRIMARY KEY);
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE lab_slot (slot_day CHAR(3), slot_time VARCHAR(11),
-> PRIMARY KEY (slot_day, slot_time));
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE grade_scale (grade CHAR(2) PRIMARY KEY, min_marks INT, max_marks INT);
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO lab_batch VALUES ('B1'), ('B2'), ('B3');
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO lab_slot VALUES ('Mon', '10:00-12:00'), ('Wed', '14:00-16:00');
Query OK, 2 rows affected (0.00 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO grade_scale VALUES ('O', 90, 100), ('A+', 80, 89), ('A', 70, 79),
-> ('B+', 60, 69), ('B', 50, 59), ('C', 40, 49), ('F', 0, 39);
Query OK, 7 rows affected (0.00 sec)
Records: 7 Duplicates: 0 Warnings: 0
```

Output of Q4: INNER JOIN of two tables: students with their department

```
mysql> SELECT s.roll_no, s.student_name, d.dept_code, d.dept_name
-> FROM student s
-> INNER JOIN department d ON s.dept_id = d.dept_id
-> ORDER BY d.dept_code, s.roll_no;
```

roll_no	student_name	dept_code	dept_name
S24AI001	Rohan Mehta	AIML	AI and Machine Learning
S24AI002	Priya Nair	AIML	AI and Machine Learning
S24AI003	Kunal Verma	AIML	AI and Machine Learning
S24AI004	Ananya Desai	AIML	AI and Machine Learning
S24CS001	Aarav Sharma	CSE	Computer Engineering
S24CS002	Isha Gupta	CSE	Computer Engineering
S24CS003	Yash Jadhav	CSE	Computer Engineering
S24EC001	Sneha Iyer	ECE	Electronics and Telecomm
S24EC002	Aditya Rao	ECE	Electronics and Telecomm
S24ME001	Neha Kulkarni	MECH	Mechanical Engineering

10 rows in set (0.00 sec)

Output of Q5: INNER JOIN of three tables: marks of students in Database Management Systems

```
mysql> SELECT s.roll_no, s.student_name, c.title, e.marks, e.grade
-> FROM enrollment e
-> JOIN student s ON s.student_id = e.student_id
-> JOIN course c ON c.course_id = e.course_id
-> WHERE c.course_id = 'CS301'
-> ORDER BY e.marks DESC;
```

roll_no	student_name	title	marks	grade
S24CS003	Yash Jadhav	Database Management Systems	95	O
S24CS001	Aarav Sharma	Database Management Systems	91	O
S24CS002	Isha Gupta	Database Management Systems	74	A
S24AI001	Rohan Mehta	Database Management Systems	70	A

4 rows in set (0.00 sec)

Output of Q6: LEFT OUTER JOIN: all departments with their student count; students with no enrollment

```
mysql> SELECT d.dept_code, COUNT(s.student_id) AS no_of_students
-> FROM    department d
-> LEFT JOIN student s ON s.dept_id = d.dept_id
-> GROUP BY d.dept_id, d.dept_code
-> ORDER BY d.dept_id;
```

```
+-----+-----+
| dept_code | no_of_students |
+-----+-----+
| CSE       | 3              |
| AIML      | 4              |
| ECE       | 2              |
| MECH      | 1              |
| CIVIL     | 0              |
+-----+-----+
```

5 rows in set (0.00 sec)

```
mysql> SELECT s.roll_no, s.student_name, e.course_id
-> FROM    student s
-> LEFT JOIN enrollment e ON e.student_id = s.student_id
-> WHERE    e.student_id IS NULL;
```

```
+-----+-----+-----+
| roll_no | student_name | course_id |
+-----+-----+-----+
| S24AI004 | Ananya Desai | NULL      |
+-----+-----+-----+
```

1 row in set (0.00 sec)

Output of Q7: RIGHT OUTER JOIN: every course with its instructor (unassigned courses included)

```
mysql> SELECT c.course_id, c.title, i.instr_name
-> FROM    instructor i
-> RIGHT JOIN course c ON c.instr_id = i.instr_id
-> ORDER BY c.course_id;
```

```
+-----+-----+-----+
| course_id | title                                | instr_name |
+-----+-----+-----+
| AI201     | Python for Data Science             | Sneha Deshmukh |
| AI301     | Machine Learning                    | Meera Joshi    |
| AI401     | Deep Learning                       | Pooja Kale     |
| CS101     | Programming in C                    | Rahul Patil    |
| CS201     | Data Structures                     | Rahul Patil    |
| CS301     | Database Management Systems         | Anil Kulkarni  |
| CS401     | Advanced Databases                  | Anil Kulkarni  |
| EC201     | Digital Electronics                 | Vikram Shinde  |
| EC301     | Signals and Systems                 | NULL          |
| ME201     | Engineering Mechanics               | Kavita More     |
+-----+-----+-----+
```

10 rows in set (0.00 sec)

```
mysql> SELECT c.course_id, c.title
-> FROM    enrollment e
-> RIGHT JOIN course c ON c.course_id = e.course_id
-> WHERE    e.course_id IS NULL;
```

```
+-----+-----+
| course_id | title                                |
+-----+-----+
| ME201     | Engineering Mechanics               |
+-----+-----+
```

1 row in set (0.00 sec)

Output of Q8: FULL OUTER JOIN: not supported by MySQL, emulated with UNION

```
mysql> SELECT i.instr_name, c.course_id
-> FROM   instructor i FULL OUTER JOIN course c ON c.instr_id = i.instr_id;
ERROR 1064 (42000) at line 191: You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near 'FULL OUTER
JOIN course c ON c.instr_id = i.instr_id' at line 2

mysql> SELECT i.instr_name, c.course_id
-> FROM   instructor i LEFT JOIN course c ON c.instr_id = i.instr_id
-> UNION
-> SELECT i.instr_name, c.course_id
-> FROM   instructor i RIGHT JOIN course c ON c.instr_id = i.instr_id
-> ORDER BY instr_name IS NULL, instr_name, course_id;

+-----+-----+
| instr_name | course_id |
+-----+-----+
| Anil Kulkarni | CS301 |
| Anil Kulkarni | CS401 |
| Kavita More | ME201 |
| Meera Joshi | AI301 |
| Nitin Pawar | NULL |
| Pooja Kale | AI401 |
| Rahul Patil | CS101 |
| Rahul Patil | CS201 |
| Sneha Deshmukh | AI201 |
| Vikram Shinde | EC201 |
| NULL | EC301 |
+-----+-----+
11 rows in set (0.00 sec)
```

Output of Q9: SELF JOIN: instructor with mentor; pairs of students from the same city

```
mysql> SELECT i.instr_name AS instructor, i.designation,
->      m.instr_name AS mentor
-> FROM   instructor i
-> LEFT JOIN instructor m ON i.mentor_id = m.instr_id
-> ORDER BY i.instr_id;

+-----+-----+-----+
| instructor | designation | mentor |
+-----+-----+-----+
| Anil Kulkarni | Professor | NULL |
| Meera Joshi | Professor | Anil Kulkarni |
| Rahul Patil | Asst. Prof | Anil Kulkarni |
| Sneha Deshmukh | Assoc. Prof | Meera Joshi |
| Vikram Shinde | Professor | Anil Kulkarni |
| Pooja Kale | Asst. Prof | Sneha Deshmukh |
| Nitin Pawar | Asst. Prof | Vikram Shinde |
| Kavita More | Assoc. Prof | Vikram Shinde |
+-----+-----+-----+
8 rows in set (0.01 sec)

mysql> SELECT a.student_name AS student_1, b.student_name AS student_2, a.city
-> FROM   student a
-> JOIN   student b ON a.city = b.city AND a.student_id < b.student_id
-> ORDER BY a.city, student_1;

+-----+-----+-----+
| student_1 | student_2 | city |
+-----+-----+-----+
| Aditya Rao | Ananya Desai | Mumbai |
| Isha Gupta | Aditya Rao | Mumbai |
| Isha Gupta | Ananya Desai | Mumbai |
| Aarav Sharma | Rohan Mehta | Pune |
| Aarav Sharma | Kunal Verma | Pune |
| Aarav Sharma | Neha Kulkarni | Pune |
| Kunal Verma | Neha Kulkarni | Pune |
| Rohan Mehta | Kunal Verma | Pune |
| Rohan Mehta | Neha Kulkarni | Pune |
+-----+-----+-----+
9 rows in set (0.00 sec)
```

Output of Q10: CROSS JOIN: all lab batch and time-slot combinations

```
mysql> SELECT b.batch, s.slot_day, s.slot_time
-> FROM lab_batch b
-> CROSS JOIN lab_slot s
-> ORDER BY b.batch, s.slot_day;
```

```
+-----+-----+-----+
| batch | slot_day | slot_time |
+-----+-----+-----+
| B1    | Mon     | 10:00-12:00 |
| B1    | Wed     | 14:00-16:00 |
| B2    | Mon     | 10:00-12:00 |
| B2    | Wed     | 14:00-16:00 |
| B3    | Mon     | 10:00-12:00 |
| B3    | Wed     | 14:00-16:00 |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

Output of Q11: Other forms: JOIN ... USING, NATURAL JOIN and a non-equi (theta) join

```
mysql> SELECT student_id, course_id, title, marks
-> FROM enrollment JOIN course USING (course_id)
-> WHERE student_id = 4;
```

```
+-----+-----+-----+-----+
| student_id | course_id | title | marks |
+-----+-----+-----+-----+
| 4 | AI201 | Python for Data Science | 92 |
| 4 | AI301 | Machine Learning | 76 |
| 4 | AI401 | Deep Learning | 84 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT dept_id, dept_code, course_id, title
-> FROM department NATURAL JOIN course
-> WHERE dept_code = 'ECE';
```

```
+-----+-----+-----+-----+
| dept_id | dept_code | course_id | title |
+-----+-----+-----+-----+
| 30 | ECE | EC201 | Digital Electronics |
| 30 | ECE | EC301 | Signals and Systems |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT s.student_name, e.course_id, e.marks, g.grade AS computed_grade
-> FROM enrollment e
-> JOIN student s ON s.student_id = e.student_id
-> JOIN grade_scale g ON e.marks BETWEEN g.min_marks AND g.max_marks
-> WHERE e.course_id = 'CS101'
-> ORDER BY e.marks DESC;
```

```
+-----+-----+-----+-----+
| student_name | course_id | marks | computed_grade |
+-----+-----+-----+-----+
| Isha Gupta | CS101 | 88 | A+ |
| Neha Kulkarni | CS101 | 78 | A |
| Aarav Sharma | CS101 | 78 | A |
| Rohan Mehta | CS101 | 65 | B+ |
| Sneha Iyer | CS101 | 55 | B |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

10. Result

All join queries were executed successfully on *college_db*. The INNER JOIN returned only matching rows (10 students with departments, 4 students in CS301); LEFT JOIN showed the CIVIL department with 0 students and student S24AI004 with no enrollment; RIGHT JOIN listed course EC301 without an instructor and ME201 without students; FULL OUTER JOIN was emulated using UNION after MySQL rejected the FULL OUTER JOIN syntax; SELF JOIN produced the mentor of every instructor and same-city student pairs; and CROSS JOIN produced $3 \times 2 = 6$ batch-slot rows.

11. Conclusion

Joins reassemble the information that normalization splits into separate tables. The choice of join decides how unmatched rows are treated: inner joins drop them, outer joins keep them with NULLs, and the anti-join pattern (LEFT JOIN ... IS NULL) finds them. Self joins handle recursive relationships such as mentoring, and cross joins generate all combinations. MySQL lacks FULL OUTER JOIN, which must be written as the UNION of a left and a right join.